

Ensembles and random forests

Forest CoverType · many models, and what averaging can and cannot do

LECTURE 7

GÉRON CH. 6

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

The previous lecture built a decision tree on this same dataset — 60,000 patches of forest, 54 cartographic columns, seven species, split 48,000 / 12,000.

Forest CoverType	Test accuracy	Conditions per justification
Always “Lodgepole Pine”	48.8%	0
Depth-8 tree, minimum leaf 20	73.0%	7.97
Unconstrained tree	82.6%	17.88

Nothing on that table is wrong. It is also not the whole story, and the missing part is what today repairs.

The one result to carry forward

MEASURED AT THE END OF THE PREVIOUS LECTURE

Refit the same depth-8 tree, with the same hyperparameters and the same seed, on twenty different 90% subsamples of the same training rows. Test accuracy moves by a standard deviation of **0.25 points** — and **9.1%** of the individual predictions change. Only **71.4%** of test patches get the same answer from all twenty.

A stable metric, an unstable model. In the language of Lecture 5's decomposition, this is **variance**: sensitivity of the fitted function to the particular training sample.

✓ **The whole of today follows from one question: what does *averaging* do to variance?**

Today

1. Where many different models come from, and why you would want them (10 min)
2. **The mathematics** — the variance of an average of n correlated predictors (20 min)
3. **The method** — voting, bagging, out-of-bag evaluation, random forests, extra-trees, feature importance (40 min)
4. Boosting, the histogram-based implementations, and stacking (15 min)

Block 2 is three lines of algebra and it predicts the shape of every measurement in block 3. It also declines to predict one of them, which is the most instructive slide of the lecture.

FIRST TEN MINUTES

Many models, and where to get them

10 minutes

The idea

Ask a difficult question of a thousand random people and aggregate their answers. The aggregate is often better than an expert's. The same is true of models, and it has a name: an **ensemble**.

A group of weak learners can form a strong learner — provided there are enough of them and they are sufficiently diverse.

“Sufficiently diverse” is doing all the work in that sentence, and it is the informal name of a quantity we are about to define. Twenty minutes from now it will have a symbol, a formula and a measured value on this dataset.

Where do many different models come from?

CART is deterministic: same data, same tree. To get twenty different trees you must give them twenty different things to look at.

Source of difference	Name
Different training algorithms on the same data	a voting classifier
The same algorithm on different random <i>subsets of rows</i> , drawn with replacement	bagging
... drawn without replacement	pasting
... plus random subsets of <i>columns</i> at each split	random forest
... plus random <i>thresholds</i>	extra-trees

The list is in increasing order of how hard it works to make two members unlike each other. That ordering is not an accident, and the next block says exactly what it is for.

Why this lecture follows the one on trees

- Averaging attacks **variance**, and trees are the high-variance model we have just measured: 9.1% of predictions move when 10% of the rows do
- A tree is cheap to fit, so two hundred of them are affordable
- An unconstrained tree has **low bias** — and averaging does nothing whatever to bias, so the members had better not have any to spare

That third point is why the ensembles in this lecture use *unconstrained* members. It looks reckless after a lecture spent limiting depth, and by the end of the next block it will look obvious.

THE MATHEMATICS

The variance of an average of correlated predictors

One result. It explains bagging, the feature subsampling in random forests, and the random thresholds in extra-trees
— all three

20 minutes · examinable

The question

Forget trees for ten minutes.

You have n predictors of the same quantity. Each has variance σ^2 . You average them.

$$\bar{f} = \frac{1}{n} \sum_{t=1}^n f_t$$

What is $\text{Var}(\bar{f})$?

Step 1 • The answer everyone gives first

$$\text{Var}\left(\frac{1}{n} \sum_t f_t\right) = \frac{1}{n^2} \sum_t \text{Var}(f_t) = \frac{\sigma^2}{n}$$

X That middle step is valid only when the f_t are uncorrelated.

The variance of a sum is the sum of the variances *plus twice the sum of the covariances*. Dropping the covariances is an assumption, and it is exactly the assumption that fails for an ensemble of trees grown on the same dataset.

Step 2 • Start with two

With $\text{Var}(f_1) = \text{Var}(f_2) = \sigma^2$ and $\text{Corr}(f_1, f_2) = \rho$:

$$\text{Var}\left(\frac{f_1 + f_2}{2}\right) = \frac{1}{4} [\sigma^2 + \sigma^2 + 2\rho\sigma^2] = \frac{\sigma^2(1 + \rho)}{2}$$

- $\rho = 0$: variance $\sigma^2/2$ — the familiar halving
- $\rho = 1$: variance σ^2 — the two predictors are the same predictor, and averaging achieves nothing

The general case has the same two limits in it. It is worth seeing them here first, where the algebra is one line.

Step 3 • Now n of them

Assume the n predictors are **identically distributed** — each with variance σ^2 — and have a common pairwise correlation

$$\text{Corr}(f_s, f_t) = \rho \quad \text{for all } s \neq t$$

Equivalently $\text{Cov}(f_s, f_t) = \rho\sigma^2$, since both have the same variance.

✓ **Identically distributed, *not* independent. That is the whole point: n trees grown on bootstrap samples of one dataset have the same distribution and are certainly not independent — they were handed the same dataset.**

Step 4 • Expand the variance of the sum

$$\text{Var} \left(\sum_{t=1}^n f_t \right) = \sum_{s=1}^n \sum_{t=1}^n \text{Cov}(f_s, f_t)$$

There are n^2 terms in that double sum. Split them by whether $s = t$:

- n diagonal terms, each $\text{Cov}(f_t, f_t) = \sigma^2$
- $n(n - 1)$ off-diagonal terms, each $\rho\sigma^2$

That is the only combinatorial step in the derivation, and it is the one worth getting right on paper.

Step 5 • Collect, and divide by n^2

$$\text{Var}(\bar{f}) = \frac{1}{n^2} \left[n\sigma^2 + n(n-1)\rho\sigma^2 \right] = \frac{\sigma^2}{n} + \frac{n-1}{n}\rho\sigma^2$$

Write $\frac{n-1}{n} = 1 - \frac{1}{n}$ and gather the two $1/n$ terms:

$$\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{n}$$

THE VARIANCE OF AN AVERAGE OF N IDENTICALLY DISTRIBUTED PREDICTORS WITH PAIRWISE CORRELATION P

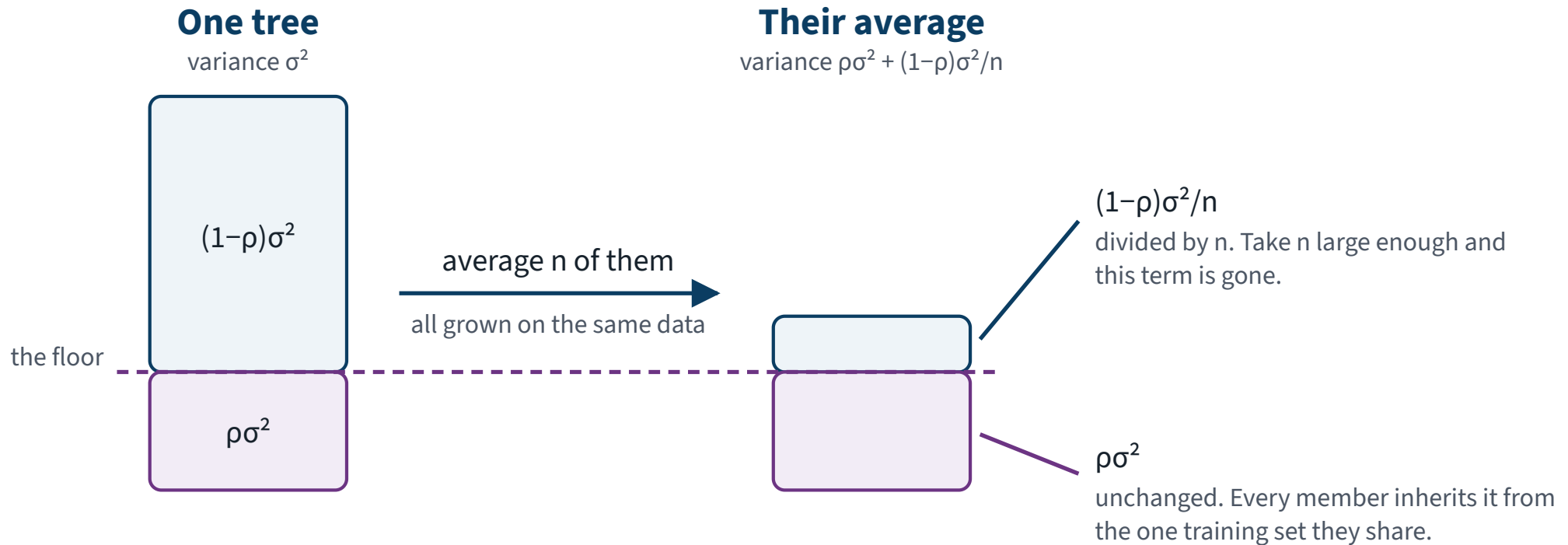
Step 6 • Read the two terms

Term	Depends on n ?	What it is
$\rho\sigma^2$	X no	the part every predictor shares
$(1 - \rho)\sigma^2/n$	yes, like $1/n$ ✓	the part each predictor has to itself

Averaging destroys the independent component and leaves the correlated one **completely untouched**.

Note what this does *not* say. It says nothing about bias. An average of n biased predictors is exactly as biased as one of them — which is why the members will be unconstrained trees.

The whole derivation, in one picture



Everything above the dashed line can be averaged away. Nothing below it can.

Three limits worth memorising

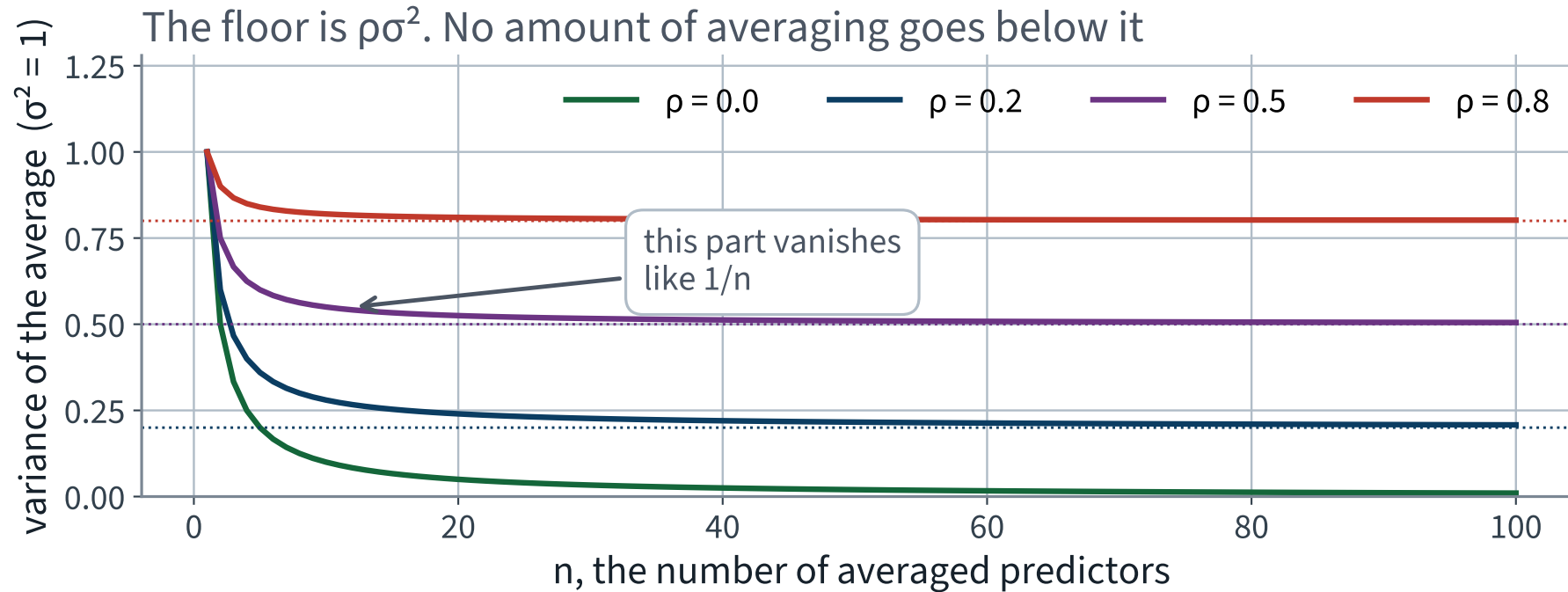
Case	Variance of the average	Reading
$\rho = 0$	σ^2/n	independent: the classical result
$\rho = 1$	σ^2	identical predictors: averaging is a no-op
$n \rightarrow \infty$	$\rho\sigma^2$	the floor — and it is reached quickly

Because the second term falls like $1/n$, most of the available reduction happens in the first few predictors. Going from $n = 1$ to $n = 10$ removes 90% of the independent part; going from 100 to 1,000 removes 0.9% more.

FAILURE CONDITION — ADDING MEMBERS TO A CORRELATED ENSEMBLE

The variance tends to $\rho\sigma^2$, so the correlated part is a floor that no number of members can touch. An ensemble that has stopped improving is not too small — it is reporting that its members are too alike, and the answer is decorrelation, not count.

The same formula, plotted



Four curves, four floors. The floor is set entirely by ρ — which is why every ensemble variant you are about to meet is an attempt to lower it.

Step 7 • What this says before we build anything

1. Averaging many models reduces variance, but only the part they do not share
2. The return on more members collapses quickly. There is little point past the point where $(1 - \rho)\sigma^2/n \ll \rho\sigma^2$
3. Therefore the way to improve an ensemble is not more members. It is **less correlated** members
4. And anything that lowers ρ by making individual members worse is a trade, not a free lunch: σ^2 appears in *both* terms, so it can rise while ρ falls

✓ **Point 4 is the one that gets forgotten, and it is the one we will measure — and be embarrassed by — in the third block.**

What that derivation buys you

EXAMINABLE

1 $\text{Var}(\sum_t f_t) = \sum_s \sum_t \text{Cov}(f_s, f_t), n^2$ terms

the definition, before any assumption

2 n diagonal at σ^2 , $n(n - 1)$ off-diagonal at $\rho\sigma^2$

identically distributed, common pairwise ρ

3 $\text{Var}(\bar{f}) = \rho\sigma^2 + (1 - \rho)\sigma^2/n$

divide by n^2 and gather the $1/n$ terms

ONE FORMULA, THREE ALGORITHMS

Bagging randomises the rows. A **random forest** also randomises the columns. **Extra-trees** also randomises the thresholds. All three exist for one purpose: to lower ρ , because $\rho\sigma^2$ is the floor that n cannot reach below.

THE METHOD

Voting, bagging, forests, and what they cost

40 minutes · examinable

Voting — the simplest ensemble

Combine *different* algorithms fitted to the same data.

```
from sklearn.ensemble import VotingClassifier

vote = VotingClassifier(
    [
        ("tree", DecisionTreeClassifier(random_state=42)),
        ("logreg", LogisticRegression(max_iter=1000)),
        ("knn", KNeighborsClassifier(n_neighbors=5))
    ], voting="soft")
```

- `voting="hard"` — majority of the predicted labels
- `voting="soft"` — average of the predicted probabilities; usually better, because a confident member outweighs several hesitant ones

Different algorithms make different kinds of errors, which is the informal way of saying ρ is small. The formula applies unchanged.

Bagging

Bootstrap aggregating. Draw m rows from the training set *with replacement*, fit a tree, and repeat n times.

- Each bootstrap sample has the same size as the training set, but about **63%** of its distinct rows — the rest are duplicates
- Aggregate by majority vote for classification, by the mean for regression
- Each member is individually worse than one tree fitted to the whole set. The ensemble is better — that is the formula, not a paradox

Pasting is the same thing without replacement. It produces less diverse members, so bagging usually wins — but it is a cheap thing to cross-validate rather than assume.

Bagging, in code

```
from sklearn.ensemble import BaggingClassifier

bag = BaggingClassifier(
    DecisionTreeClassifier(random_state=42),    # unconstrained members
    n_estimators=200, bootstrap=True, oob_score=True,
    random_state=42, n_jobs=-1)

bag.fit(X_train, y_train)
print(f"out-of-bag {bag.oob_score_:.1%}")
```

The members are **unconstrained** trees. We want each one to have low bias, because averaging cannot remove bias — and we are about to average the variance away, so there is no reason to restrict their depth.

A validation set you get for nothing

Each member never sees about 37% of the rows — its **out-of-bag** instances. Score each member on its own out-of-bag rows and you have a generalisation estimate with no validation split and no extra fit.

WHY 37%

A given row is missed by one draw with probability $1 - 1/m$, so by all m draws with probability $\left(1 - \frac{1}{m}\right)^m \rightarrow e^{-1} \approx 0.368$ as m grows. Hence about 63% distinct rows in, 37% held out.

Cheap, and it does not replace the test set. It replaces the *validation* set. If out-of-bag comes out much better than the test score, something in your pipeline saw the out-of-bag rows anyway.

Reading an out-of-bag score

- It is an average over members, each scored on a *different* 37% of the rows — so it estimates the generalisation of one member's procedure, not of the ensemble you ship
- It is available only where the members see bootstrap samples. With `bootstrap=False` — the extra-trees default — there are no out-of-bag rows and `oob_score=True` raises
- Use it to compare configurations cheaply, then confirm the one you pick with cross-validation or the test set

WHEN IT BITES

If the out-of-bag score comes out much *better* than the test score, something in the pipeline saw the out-of-bag rows anyway — a scaler fitted outside the ensemble, or a feature built from the whole frame. The gap is a leakage detector you get for free.

Random forests

Bagging, plus one more source of difference: at **every node of every tree**, only a random subset of the features is considered for the split.

```
from sklearn.ensemble import RandomForestClassifier

rnd = RandomForestClassifier(n_estimators=200, max_features="sqrt",
                             random_state=42, n_jobs=-1).fit(X_train, y_train)
```

`max_features="sqrt"` is the classification default: $\lfloor \sqrt{54} \rfloor = 7$ of the 54 columns, drawn afresh at each node. A tree with 200 nodes therefore draws 200 different subsets.

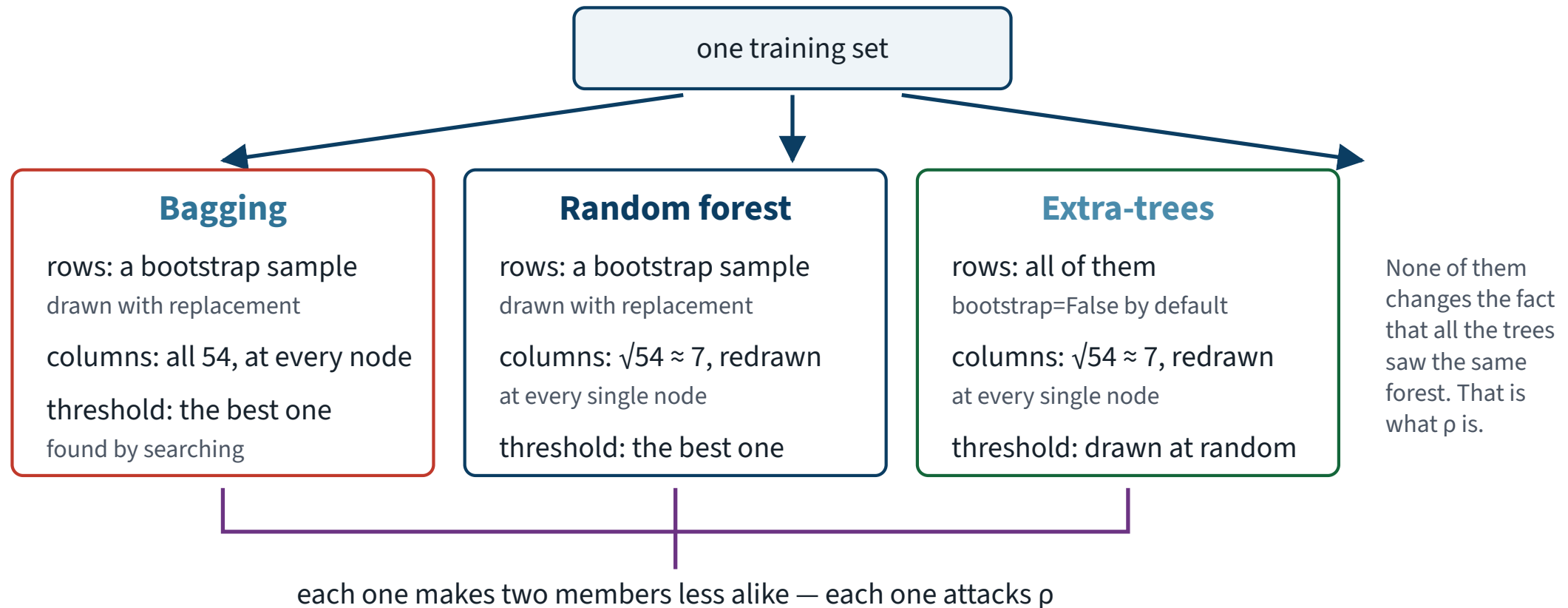
Extra-trees

One further step. Having chosen the candidate features, do not search for the best threshold — **draw one at random** for each candidate and keep the best of those.

- Much faster: the threshold search is the expensive part of training a tree
- More diverse members, each individually worse
- Scikit-Learn's default is `bootstrap=False` — the random thresholds *replace* the bootstrap rather than joining it

X That default is a decision somebody else made, it is not what most descriptions of extra-trees imply, and we are going to measure what it costs rather than argue about it.

What each variant randomises



Rows, columns, thresholds. Three knobs, and all three are turned for the same reason.

The claim this lecture is making

ONE FORMULA, THREE ALGORITHMS

Bagging randomises the **rows**. Random forests also randomise the **columns**. Extra-trees also randomise the **thresholds**. All three exist to reduce one quantity: ρ , the correlation term in $\text{Var}(\bar{f}) = \rho\sigma^2 + (1 - \rho)\sigma^2/n$.

That is the payoff of the derivation, and it is the thing to remember if you remember one thing from this lecture.

X It is also a claim, and this course does not accept claims. Measure it.

Measuring ρ needs something we do not have

ρ is the correlation between two members over the randomness of the *whole procedure* — which includes which training set you were handed. Two trees grown on one dataset are correlated **through that dataset**.

So a single training set cannot show it. Conditional on the data, the members are independent by construction and ρ would measure as zero.

WHAT THE EXPERIMENT NEEDS

Several *independent* training sets. CoverType has 581,012 rows, so we can cut **20 disjoint training sets of 20,000 rows** — 400,000 rows in all, no row reused — and still keep 12,000 rows aside as a shared test set.

The design

```
for k in range(20):                                # 20 disjoint training sets
    idx = pool[k * 20_000:(k + 1) * 20_000]
    model = make_ensemble(kind, n=20, seed=1000 + k).fit(X[idx], y[idx])
    for j, member in enumerate(model.estimators_):
        S[k, j] = (classes[member.predict(X_test)] == y_test)
# S has shape (20 training sets, 20 members, 12,000 test patches)
```

$S_{k,j,i} = 1$ when member j of ensemble k is right about patch i . Two indices of randomness: *which* training set, and *which* member given the set. That is exactly the split the formula needs.

Splitting the variance in two

At each test patch, decompose the variance of a single member across all 400 fitted trees:

$$\sigma^2 = \underbrace{\tau^2}_{\text{between training sets}} + \underbrace{w^2}_{\text{within a training set}} \quad \rho = \frac{\tau^2}{\tau^2 + w^2}$$

- w^2 : how much two members of the *same* ensemble differ
- τ^2 : how much the ensembles differ from one another — the part every member of a given ensemble inherits from its data
- τ^2 *is* the floor $\rho\sigma^2$. Not a coincidence; it is the same quantity written twice

One trap in that code, worth ten minutes of your life

```
>>> rnd.classes_                                # the ensemble's labels
array([1, 2, 3, 4, 5, 6, 7])
>>> rnd.estimators_[0].classes_                 # one member's labels
array([0., 1., 2., 3., 4., 5., 6.])
```

WHEN IT BITES

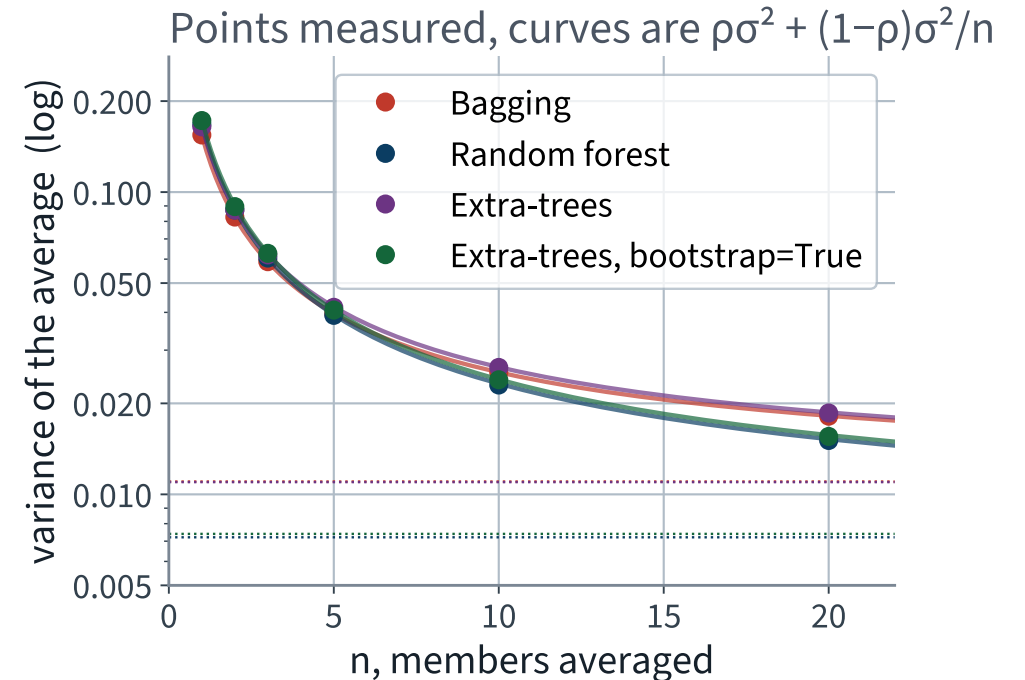
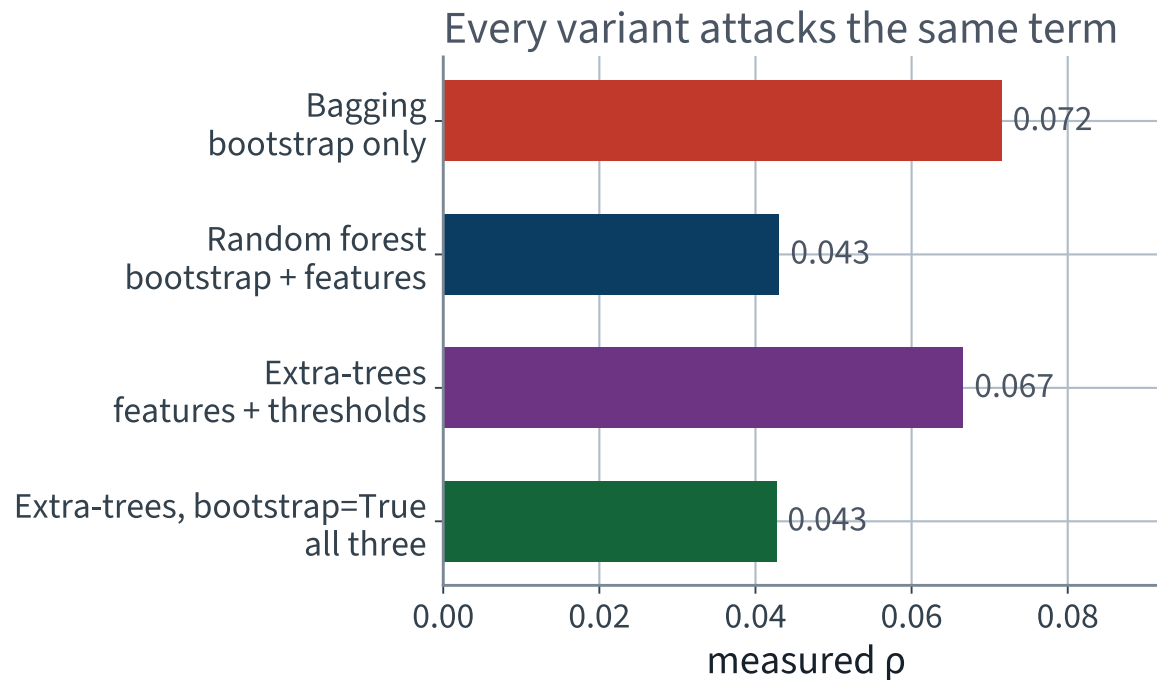
Every ensemble re-encodes the labels as positions before handing them to its members, so a member's `predict` returns **positions**, not cover types. Compare them with `y_test` directly and every member scores about 8% — which looks like a terrible model rather than a bug.

✓ `classes_[member.predict(X)]` maps it back. Print the two arrays side by side any time you reach inside an ensemble.

It is a library convention, not a property of the method — but it costs an afternoon the first time.

NOT EXAMINABLE — ENGINEERING

ρ , measured on four ensembles



The points are measured and the curves are $\rho\sigma^2 + (1 - \rho)\sigma^2/n$. Nothing was fitted to make them line up.

First: does the formula actually hold?

Ensemble	σ^2	ρ	Var of the average of 20, measured	... predicted
Bagging	0.1542	0.072	0.01814	0.01819
Random forest	0.1675	0.043	0.01508	0.01523
Extra-trees	0.1648	0.067	0.01859	0.01867
Extra-trees, <code>bootstrap=True</code>	0.1735	0.043	0.01554	0.01570

✓ **Measured and predicted agree in the third decimal place, on all four. The derivation from twenty minutes ago is a description of this dataset.**

Agreement this good is partly luck — the formula assumes every pair has the same ρ , which is only approximately true. Do not expect three decimals every time.

Second: does each variant lower ρ ?

Ensemble	Rows	Columns	Thresholds	ρ
Bagging	bootstrap	all 54	best	0.072
Extra-trees (default)	all	7 of 54	random	0.067
Random forest	bootstrap	7 of 54	best	0.043 ✓
Extra-trees, <code>bootstrap=True</code>	bootstrap	7 of 54	random	0.043 ✓

Every variant is below bagging, so the claim survives. But read the last two rows: adding random thresholds on top of a random forest moves ρ from 0.043 to 0.043. **The mechanisms are not additive.**

The honest reading of that table

- **Feature subsampling is doing the work here.** It cuts ρ by 40%, from 0.072 to 0.043
- **Random thresholds are a substitute for the bootstrap, not an addition to it.** Extra-trees without a bootstrap lands at 0.067, near bagging; with one, at 0.043, exactly where the forest is
- **There is a floor to the floor.** Nothing we tried gets below 0.043, because all four ensembles ultimately saw the same 20,000 rows

That last point is the formula's real message. ρ is not a property of the algorithm. It is the share of the variance that comes from *which data you were given*, and no amount of algorithmic randomness reaches it.

How much variance did the averaging remove?

Ensemble	One member	Average of 20	Removed	Floor $\rho\sigma^2$
Bagging	0.15453	0.01814	88.3%	0.01103
Random forest	0.16734	0.01508	91.0% ✓	0.00721 ✓
Extra-trees	0.16439	0.01859	88.7%	0.01098
Extra-trees, <code>bootstrap=True</code>	0.17248	0.01554	91.0%	0.00740

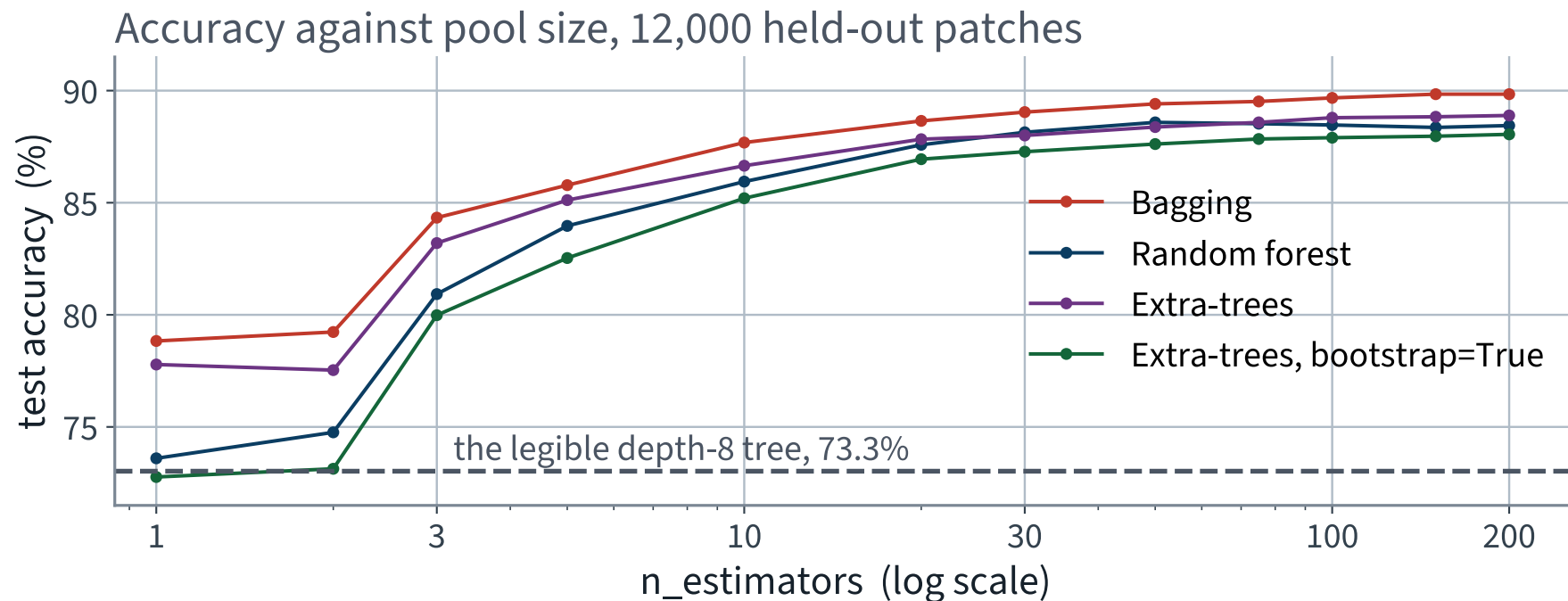
Twenty members remove about nine tenths of a single member's variance. Going to two hundred would remove roughly one twentieth more, and then stop — exactly what $(1 - \rho)\sigma^2/n$ says.

And what did that do to the accuracy?

200 members, 48,000 training patches	10 members	200 members
Bagging	87.7%	89.8% ✓
Extra-trees	86.7%	88.9%
Random forest	85.9%	88.4%
Extra-trees, <code>bootstrap=True</code>	85.2%	88.0%
The legible depth-8 tree	—	73.0%
Always “Lodgepole Pine”	—	48.8%

✓ Every ensemble beats the constrained tree by at least fifteen points, and the *unconstrained* tree by at least five. The last 190 members buy about two.

How many members do you actually need?



Most of the gain arrives by thirty members, exactly as $(1 - \rho)\sigma^2/n$ says it should.

Now look at that ranking again

Ensemble	ρ	One member's accuracy	Ensemble accuracy
Bagging	X 0.072	78.8% ✓	89.8% ✓
Extra-trees	0.067	77.8%	88.9%
Random forest	0.043 ✓	73.6%	88.4%
Extra-trees, <code>bootstrap=True</code>	0.043 ✓	X 72.8%	X 88.0%

~~X~~ The ensemble with the *highest* ρ is the most accurate.

That is not a contradiction — read the formula again

$$\text{Var}(\bar{f}) = \rho \sigma^2 + \frac{(1 - \rho) \sigma^2}{n}$$

- The formula is about **variance**. It says nothing about bias, and accuracy depends on both
- Every mechanism that lowers ρ does so by **handicapping the members**. A tree allowed only 7 of 54 columns per node is a worse tree: 73.6% against bagging's 78.8%
- σ^2 appears in *both* terms. Lowering ρ while raising σ^2 can leave you worse off

✓ **On this dataset — 54 columns of which perhaps a dozen carry most of the signal — throwing away 47 at every node costs more than the decorrelation is worth. On a dataset with hundreds of weakly informative columns it is usually the other way round, which is why the default exists.**

What the derivation predicted, and what it declined to

- It predicted the *shape* of the accuracy curve — steep, then flat, with the knee where $(1 - \rho)\sigma^2/n$ falls below $\rho\sigma^2$
- It predicted the variance of a 20-member average to three decimal places, on four different ensembles
- It told us in advance that more members would stop helping, and roughly when
- It did **not** tell us which ensemble would be most accurate, and it never claimed to

A theory that predicts three things and declines to predict the fourth is not a weak theory. A theory that predicts all four without being asked is usually being misread.

Now the bill

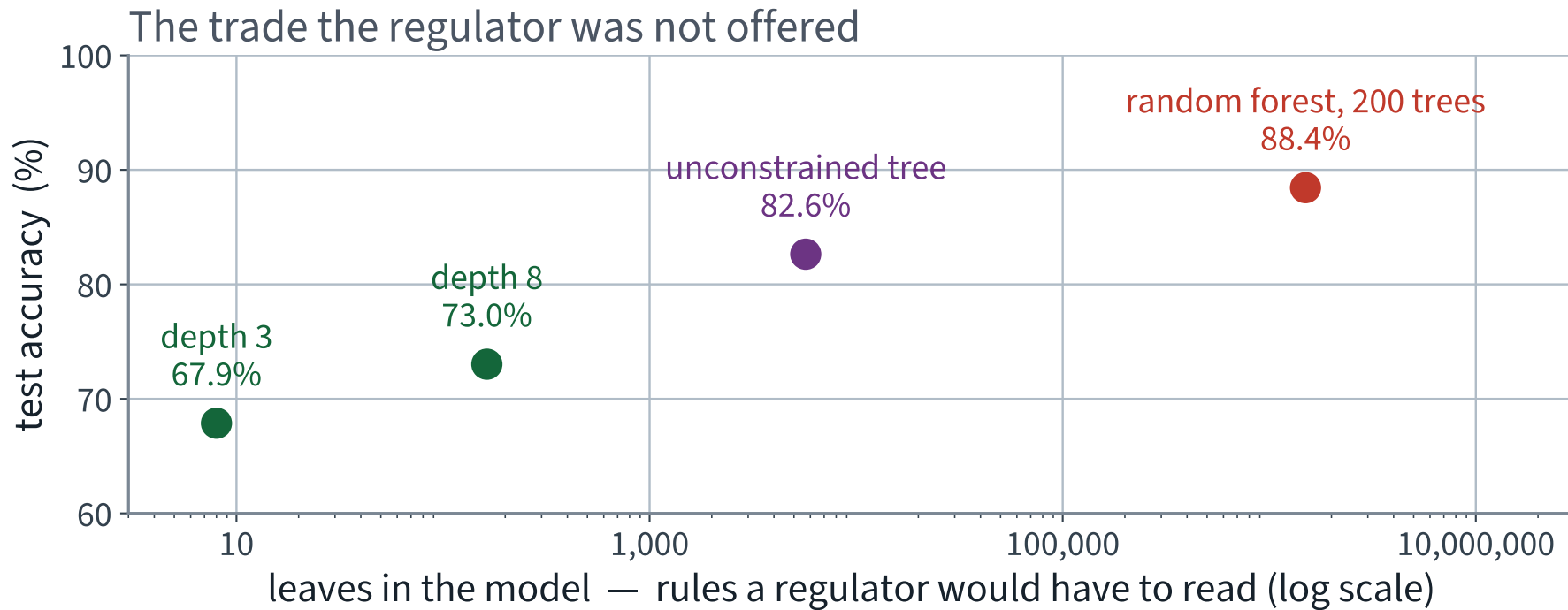
We removed the variance. Ask what happened to the thing the agency asked for.

```
total = sum(t.get_n_leaves() for t in rnd.estimators_)
print(f"{total:,} leaves across the 200 trees")
```

```
1,497,919 leaves across the 200 trees
```

X The justification for one prediction is now 200 decision paths and a vote.

The trade, drawn to scale



X Fifteen points of accuracy, and four orders of magnitude more model to read.

Do not smooth this over

FAILURE CONDITION • AN ENSEMBLE HAS NO PER-PREDICTION JUSTIFICATION

The agency asked for a model whose every prediction comes with a human-readable justification. We have built one that is 15.4 points more accurate and **cannot supply one**. There is no technical fix.

The tools in the rest of this lecture recover something, and what they recover is genuinely less than what was lost. Saying so is part of the job.

What can be recovered

The agency asked for	An ensemble can offer
Why <i>this</i> parcel	nothing comparable X
Which measurements the decision rests on, overall	feature importance ✓
How confident the model is	the vote share — and it is better calibrated than one tree's leaf proportions ✓
Reproducibility of the answer	much improved — that was the whole repair ✓

Three of four is not nothing. The one we cannot supply is the one the agency wrote down first.

What the vote share is, and is not

A single tree's `predict_proba` returns the class proportions of one leaf: piecewise constant, identical for every patch reaching that leaf, and often built on a handful of training rows.

An ensemble averages 200 such vectors, each from a different leaf of a different tree. The result varies smoothly across the feature space and is **better calibrated** — which is the one thing on the agency's list that averaging improves rather than destroys.

WHEN IT BITES

Better calibrated is not calibrated. A forest's vote share is systematically pulled towards the middle, because unanimity is rare. If a downstream decision uses the number as a probability, check it against observed frequencies before believing it.

Feature importance, as the library offers it

```
imp = pd.Series(rnd.feature_importances_, index=X_train.columns)
print(imp.sort_values(ascending=False).head(5).round(3))
```

`feature_importances_` sums, over every node that split on a feature, the weighted impurity reduction that split achieved — the ΔI of the previous lecture, accumulated.

It runs, it is fast, and the top of the list is entirely sensible: elevation first, then the distances. Nothing invites suspicion.

✓ **Before trusting any ranking, ask what it would look like on a variable you *know* carries no information. If you cannot answer, add one and find out.**

Add a control column and refit

```
rng = np.random.default_rng(42)
X_tr2 = X_train.assign(random_decoy=rng.random(len(X_train)))
X_te2 = X_test.assign(random_decoy=rng.random(len(X_test)))

rnd2 = RandomForestClassifier(n_estimators=100, max_features="sqrt",
                             random_state=42, n_jobs=-1).fit(X_tr2, y_train)
```

A column of uniform random numbers. It cannot possibly carry information about which trees grow where. Where does it rank among the 55?

Tenth of fifty-five

10 / 55

The random column ranks **above 45 of the 54 real measurements**, with an impurity importance of 0.0399 — against 0.2277 for elevation.

X Put that list in front of the agency and you have asserted that a column of noise matters more than 45 things the surveyors actually recorded.

Why impurity importance does that

FAILURE CONDITION • TWO BIASES, BOTH STRUCTURAL

It is measured on the training data. A split that reduces impurity on the rows that chose it will do so whether or not the feature is informative.

It favours high-cardinality features. A continuous column offers thousands of candidate thresholds; a 0/1 soil-type column offers one. More chances to find a lucky split means more accumulated ΔI .

Our decoy is continuous and 44 of the 54 real columns are binary. The ranking is not a mystery once you know what is being summed — and Step 6 of the previous lecture told you what is being summed.

The measurement with the control built in

Shuffle one column of the **held-out** set, re-score, and see how much accuracy falls. Repeat, and report a spread.

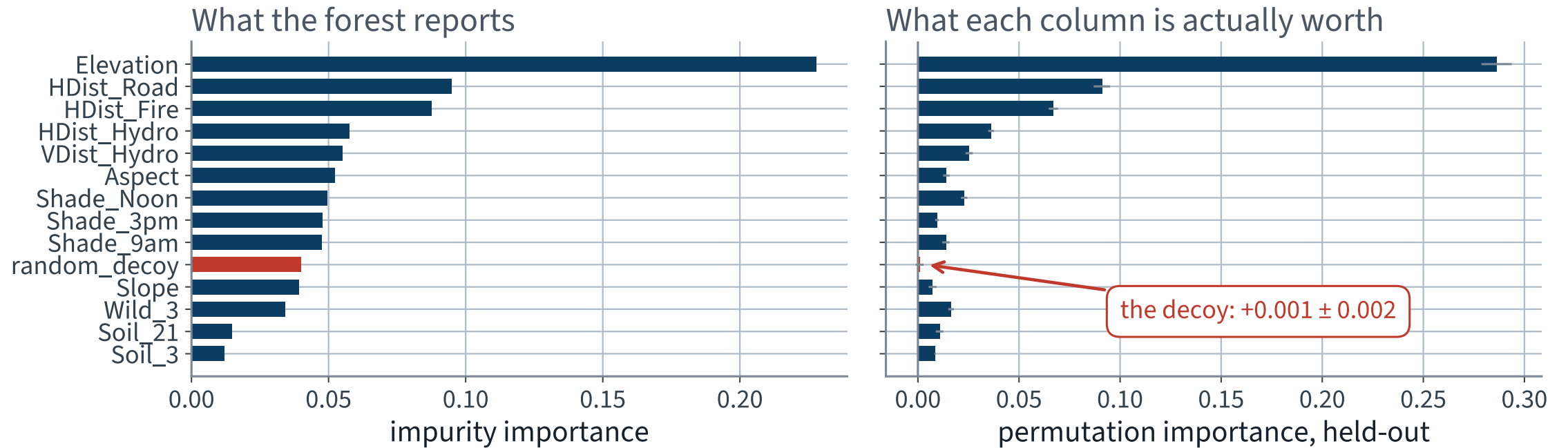
```
from sklearn.inspection import permutation_importance

r = permutation_importance(rnd2, X_te2, y_test, n_repeats=5,
                           random_state=42, n_jobs=-1)
```

SCOPE

`permutation_importance` goes beyond Chapter 6. It is here BEYOND THE SYLLABUS, FOR CONTEXT because the alternative is to ship the biased measurement, and because the idea — break the column and see what breaks — is worth ten minutes of anyone's time.

The same columns, measured two ways



✓ Same model, same columns, same ordering of the rows. The decoy is tenth on the left and zero on the right.

The decoy, both ways

Measurement	random_decoy	Elevation
Impurity importance (training data)	X 0.0399	0.2277
Rank among the 55 columns, by impurity	X 10th	1st
Permutation importance (held out)	+0.00085 ± 0.00195 ✓	far above it, and the notebook prints both

Permutation importance puts the decoy at less than half a standard deviation from zero — which is the correct answer, and which the impurity measure never had any way of reaching.

The forest with the decoy scores 87.6%, against 88.4% without. Adding a noise column costs a little accuracy and a lot of credibility.

What importance can and cannot tell the agency

Question	Can importance answer it?
Which measurements should the survey keep collecting?	yes — this is what it is for ✓
Is the model using a variable it legally must not?	yes, as a screen ✓
Why was <i>this</i> parcel refused?	no X
Would removing this column hurt?	no — remove it, refit, and measure X

X Row three is the agency's actual question, and it is the row importance cannot fill. Do not present it as though it can.

THE LAST FIFTEEN MINUTES

Boosting, and what practitioners actually use

15 minutes · examinable

Boosting — a different idea entirely

Everything so far trains members **in parallel** and averages them. Boosting trains them **in sequence**, each one correcting its predecessor.

The members are not identically distributed and are certainly not exchangeable, so **today's formula does not apply to boosting**. Bagging reduces variance; boosting reduces *bias*.

X Two consequences: boosting cannot be parallelised across members, and it is much easier to overfit with. Both follow from the sequence.

AdaBoost

1. Fit a weak learner — typically a tree of depth 1, a *decision stump*
2. **Increase the weight** of the instances it got wrong
3. Fit the next learner on the reweighted data; repeat
4. Predict by a weighted vote, each learner weighted by its own accuracy

```
from sklearn.ensemble import AdaBoostClassifier

ada = AdaBoostClassifier(DecisionTreeClassifier(max_depth=1),
                        n_estimators=200, learning_rate=0.5,
                        random_state=42)
```

If it overfits, reduce `n_estimators` or `learning_rate`, or regularise the base estimator.

Gradient boosting

Same sequence, different correction. Instead of reweighting the instances, fit each new learner to the **residual errors** of the ensemble so far.

- `learning_rate` scales each member's contribution. A low rate needs more members and generalises better — this is *shrinkage*
- Too few members underfits; too many overfits. Use early stopping with `n_iter_no_change`
- The number of members is therefore *not* a “more is better” parameter, which is exactly the opposite of bagging

WHEN IT BITES

Adding trees to a bagged ensemble can only help, because $(1 - \rho)\sigma^2/n$ falls monotonically. Adding trees to a boosted ensemble eventually hurts. Carrying the first intuition into the second is one of the commonest mistakes with these models.

What practitioners actually reach for

`HistGradientBoostingClassifier` bins each feature into at most 255 buckets before growing anything, so a split search costs $O(\text{bins})$ rather than $O(m)$.

```
from sklearn.ensemble import HistGradientBoostingClassifier

hgb = HistGradientBoostingClassifier(max_iter=100, learning_rate=0.05,
                                     early_stopping=False,
                                     random_state=42).fit(X_train, y_train)
```

- Dramatically faster on a dataset this size, and it is the same algorithm LightGBM and XGBoost implement
- Handles missing values and categorical columns natively — no imputer, no encoder
- On tabular data it is the usual first choice, and on most benchmarks it is the strongest single model

X On *this* dataset, at a budget that fits a lecture, it lands several points below the bagged forest. The notebook measures both, and measures what happens when you raise the learning rate. Usual is not always.

Stacking

Rather than voting or averaging, **train a model to do the aggregation**. The base learners' predictions become the input features of a final estimator, the *blender*.

- The blender must be trained on predictions the base learners made for instances they did not see — otherwise it is learning from memorised answers
- `StackingClassifier` does this with cross-validation internally, which is the only reason it is safe to use

WHEN IT BITES

Build stacking by hand and the blender sees in-sample predictions, which are far too good. It then learns to trust whichever base learner overfits hardest, and the whole ensemble is worse than its best member — with no error and no warning.

Which to reach for

Situation	Reach for
A high-variance base learner and time to spare	bagging or a random forest
Many weakly informative columns	a random forest — feature subsampling earns its keep
Training time is the binding constraint	extra-trees
You want the strongest tabular model	histogram gradient boosting — usually, and measure it against a forest anyway
You must justify individual predictions	X none of them — go back to one tree

The last row is the one this pair of lectures exists to teach, and it is the row people skip.

Did the averaging fix what we diagnosed?

The diagnosis was *instability*, not inaccuracy. So test the thing that was diagnosed, not the thing that is easy to measure.

Random forest, 20 disjoint training sets	Measured
Variance of one member's correctness	X 0.16734
Variance of the average of 20	0.01508 ✓
Share of the variance the averaging removed	91.0% ✓
The floor $\rho\sigma^2$ it cannot reach below	0.00721

✓ Nine tenths of a single member's variance, gone — and we knew before starting both that it would stop and roughly where. The notebook repeats the twenty-subsample disagreement measurement on a forest; predict the answer before you run it.

The standing constraint, extended

TWO CLAUSES ADDED TODAY

“... Split before anything is fitted. All preprocessing inside a `Pipeline` passed to cross-validation. Nothing derived from the test set in the training path. Fixed seeds. Per-fold scores, not just the mean. **Any importance or explanation is computed on held-out data and reported with a control. Any per-instance claim is checked for stability under refitting.**”

Both clauses come from a measurement in this lecture rather than from a principle: the decoy at rank 10 of 55, and 9.1% of predictions moving while the accuracy did not.

What is examinable from today

- The derivation in full — the double sum, the diagonal/off-diagonal split, the two terms, and the three limits
- What each of bagging, pasting, random forests and extra-trees randomises, and which term of the formula it attacks
- Out-of-bag evaluation, including why the fraction is e^{-1}
- Why lower ρ does not imply higher accuracy
- Why impurity importance is biased, in both of its two ways
- Boosting against bagging: bias against variance, sequence against parallel, and why the formula does not apply to boosting

Not examinable: `permutation_importance` itself, the ANOVA estimator used to measure ρ , library defaults, wall-clock timings.

The notebook for this lecture

[Open Lecture 7 in Colab](#)

- **Runs on free CPU.** It rebuilds the previous lecture's split from the seed, so nothing has to be carried across from that notebook. Several cells take a minute or two on two cores and say so
- Everything on today's slides, on the same rows with the same seed: the four 200-member pools, the twenty disjoint training sets, the variance decomposition, the decoy column and both importance measurements
- It checks the derivation twice — once on synthetic predictors with a ρ you choose, and once against the four measured ensembles

WHAT TO CHANGE BEFORE THE NEXT LECTURE

Set `max_features=None` on the random forest, which turns it back into bagging. Watch ρ rise, each member get better, and the ensemble accuracy rise with it — the trade of the whole lecture, in one keyword.

Every notebook in the course

01 · What machine learning is, and how we will work

02 · The end-to-end project

03 · Classification and its metrics

04 · Training models

05 · Regularisation and the bias–variance trade-off

06 · Decision trees

07 · Ensembles and random forests

08 · Dimensionality reduction and unsupervised learning

09 · Neural networks, from the perceptron up

10 · PyTorch

11 · Training deep networks

12 · Convolutional networks

13 · Transfer learning

14 · Detection and segmentation

15 · Time series

16 · Recurrent networks

17 · Text

18 · Attention and transformers

19 · Information retrieval: the lexical foundation

20 · Information retrieval: dense retrieval

21 · Recommender systems: from ratings to factors

22 · Recommender systems: neural, and evaluated honestly

23 · Vision transformers and multimodal retrieval

24 · Generation, retrieval-augmented systems, and where this leaves you

What we did

1. Derived $\text{Var}(\bar{f}) = \rho\sigma^2 + (1 - \rho)\sigma^2/n$ from the double sum of covariances: averaging destroys the independent component and cannot touch the correlated one
2. Read the two terms as a policy — more members buys less and less, and the way to improve an ensemble is *less correlated* members
3. Built bagging, random forests and extra-trees, and identified what each randomises: rows, then columns, then thresholds
4. Measured ρ on 20 disjoint training sets and found the formula predicts the variance of a 20-member average to three decimals, on four different ensembles
5. Found ρ falls from 0.072 to 0.043 — and that the mechanisms are **not additive**: random thresholds substitute for the bootstrap rather than adding to it
6. Found the lowest- ρ ensemble is *not* the most accurate, because every mechanism that lowers ρ also weakens the members, and σ^2 is in both terms
7. Bought 15.4 points of accuracy with 1,497,919 leaves — and lost the per-prediction justification the previous lecture was built around
8. Showed impurity importance ranking a column of pure noise 10th of 55, and repaired it with a permutation measurement on held-out data

In the book

Topic	Chapter
Voting classifiers, hard and soft	6
Bagging, pasting, out-of-bag evaluation	6
Random forests, extra-trees, feature importance	6
AdaBoost, gradient boosting, histogram-based boosting	6
Stacking and the blender	6
Decision trees, impurity, instability	5 (Lecture 6)
The bias–variance decomposition	4 (Lecture 5)

NOT PRESENTED — FOR AFTERWARDS

Exercises

Lecture 7

five, in the style of the written paper

Exercises — Lecture 7

- 1** For n predictors each of variance σ^2 and pairwise correlation ρ , the variance of their average is $\rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$. State the limit as $n \rightarrow \infty$, and what it means for ensembles. [5]
- 2** Extra-trees are usually faster to fit than a random forest and often no less accurate. Give the mechanism, in terms of the formula above. [4]
- 3** You set `bootstrap=False` and `oob_score=True`. State what happens and why. [3]

Solutions on Lecture 8's deck.

Exercises — Lecture 7 (continued)

- 4** Impurity-based feature importance ranks a random decoy column above several real ones. Explain how that can happen, and name the measure you would use instead. [5]
- 5** A forest improves accuracy over one tree by far less than the variance reduction would suggest. Is this a contradiction? Answer with a reason. [4]

Solutions on Lecture 8's deck.

THE FIVE SET AT THE END OF LECTURE 6

Solutions Lecture 6

not part of this lecture

Solutions — Lecture 6

1. Gini and entropy give nearly the same trees on CoverType. State the property they share that explains it, and...

✓ Both are strictly concave with a maximum at the uniform distribution and zero at a pure node; they differ only where two splits are nearly tied.

- Any impurity with those properties ranks most candidate splits identically
- They can part only on a pair of candidate splits whose children are incomparable in that ordering and whose weighted impurities differ by less than 0.055 — a close call deep in the tree, never at the root

2. A decision tree is described as ‘nonparametric’. Explain what that means here, and what it...

✓ Its structure is not fixed before fitting, so it will grow to fit the data exactly unless constrained.

- The parameter count is decided by the data, not by the model class
- Constraints such as depth or minimum leaf size are what stop it memorising

Solutions — Lecture 6 (2)

3. The brief allows at most eight conditions per decision. You measure the number of conditions as...

✓ The decision path counts the nodes visited, including the leaf, and the leaf applies no condition.

- A path of depth d visits $d + 1$ nodes
- Without the subtraction the count exceeds the tree's own depth, which is what the assert catches

4. Twenty trees are refit on 90% subsamples of one training set, so any two of them share about 80% of their...

✓ Yes, in the way that matters. The accuracy is stable to 0.25 points while 9.1% of individual predictions change — a stable metric is not evidence of a stable model.

- Accuracy averages over 12,000 patches, and an average hides substitutions: a tree that loses one patch and gains another scores the same
- The instability is deep in the tree, not at the root — the root feature is Elevation in all twenty refits — so the single decision you would be asked to explain is the part that does not reproduce

Solutions — Lecture 6 (3)

5. Per-class recall shows one class far below the rest. Give two responses that do not involve changing the...

✓ Report per-class recall beside the headline, and take the question back to the client. Report first.

- A single accuracy averages the class away; per-class recall costs nothing and makes the failure visible, which is the only response available before anyone has decided anything
- Whether that class is worth other classes' recall is the agency's decision, not a machine learning one — and reweighting, the obvious next step, is a different model answering a different question

LECTURE 8 · GÉRON CH. 7–8

Dimensionality reduction and unsupervised learning

PCA via the SVD, derived — in what sense the principal subspace minimises reconstruction error

Random projection and the Johnson–Lindenstrauss bound

Then clustering: k-means, silhouette, DBSCAN and Gaussian mixtures

Run the Lecture 7 notebook first